

Design of a Vector Embedding for Capability Composition

Assignment 2

Abstract

This assignment investigates a vector-based representation of formally specified application capabilities and how smaller capabilities can be composed into a larger workflow. A sample online-shopping application is used, with capabilities for order creation, payment, notification, cart cancellation, refunds, and delivery tracking. Capabilities are encoded as multi-hot vectors over a shared feature vocabulary containing type, inputs, outputs, preconditions, effects, and resources. Cosine similarity is used as an exploratory measure of vector relatedness, while explicit symbolic checks are used to assess execution compatibility. A composition routine aggregates external inputs, preconditions, effects, outputs, resources, and operational attributes. The implementation also simulates a three-capability order-payment-notification workflow and generates CSV, JSON, and graphical results. The prototype demonstrates a practical starting point, while its simplified feature encoding and compatibility rules identify clear opportunities for further evaluation and refinement.

1. Problem Definition

Modern applications consist of reusable operations or services. To build a larger function, a system must determine what each operation requires, what it produces, which state conditions it changes, and whether its output can satisfy another operation's requirements. The assignment focuses on representing these functional relationships in a vector space rather than designing a planning or path-search algorithm.

The sample problem is an online-shopping workflow. The initial state indicates that a user is authenticated and has a non-empty cart. The desired goal is that an order exists, payment is completed, and a notification has been sent. The selected workflow is CreateOrder → MakePayment → SendNotification.

2. Design Requirements

- Represent application states, goals, and atomic capabilities in a structured, machine-readable form.
- Encode capability type, inputs, outputs, preconditions, effects, and resource requirements.
- Represent functional relatedness with a vector comparison measure while retaining explicit checks for execution compatibility.
- Construct a composite capability from an ordered sequence of atomic capabilities.
- Consider operational attributes such as latency, reliability, risk, and cost.
- Evaluate compatibility, composition, alternatives, goal relevance, operational properties, consistency, and efficiency.
- Provide reproducible implementation outputs and document assumptions and limitations.

3. Related Embedding Approaches

Distributed word embeddings such as Word2Vec represent items as numerical vectors and can capture patterns of semantic relatedness from data. Such embeddings are useful inspiration, but capability composition differs from word similarity: two operations can be semantically or functionally similar without being executable in sequence. For this reason, the prototype uses a transparent, feature-based multi-hot representation instead of training a language model.

A multi-hot vector marks the presence of named features. It is interpretable and easy to implement, but it does not learn latent relationships and depends on a carefully designed vocabulary. Cosine similarity provides a normalized measure of vector direction, but by itself it cannot prove that preconditions, types, constraints, or data dependencies are satisfied. The design therefore combines vector similarity with symbolic compatibility and state simulation.

4. Proposed Representation

The application is described by $A = (S, C, SI, G, R, K)$, where S is the state space, C is the set of capabilities, SI is the initial state, G is the goal specification, R is the resource set, and K is the global constraints and policies.

Each capability is represented conceptually as $C_i = (T_i, I_i, O_i, P_i, E_i, K_i, R_i, Q_i, Rel_i, A_i, M_i)$. These fields denote capability type, inputs, outputs, preconditions, effects, capability-specific constraints, resources, quality attributes, reliability, availability, and execution mechanism. The implemented dataset explicitly stores type, inputs, outputs, preconditions, effects, resources, latency, reliability, risk, and cost. Constraints, availability, and detailed execution mechanisms are not yet fully represented in the current code.

A shared vocabulary F is built from observed feature labels. The embedding $\phi(C_i)$ is a binary vector in $\{0,1\}^{|F|}$. Features include type:<type>, input:<name>, output:<name>, pre:<condition>=<value>, effect:<condition>=<value>, and resource:<name>. A feature is set to 1 when it is present in the capability and 0 otherwise. Goal conditions are encoded using effect-style features so that they occupy the same vector space.

5. Mathematical Formulation

Let $F = \{f_1, f_2, \dots, f_d\}$ be the shared feature vocabulary. The capability embedding is $\phi(C) = [x_1, x_2, \dots, x_d]$, where $x_j = 1$ if feature f_j occurs in C and $x_j = 0$ otherwise.

Cosine similarity between vectors x and y is defined as $\text{sim}(x,y) = (x \cdot y) / (||x|| ||y||)$, when both vectors have non-zero norm. It measures feature overlap in a normalized vector space. In the program, cosine similarity is calculated using scikit-learn.

A symbolic compatibility relation is also required. A capability C_j is a candidate successor of C_i when C_i produces data required by C_j and/or C_i 's effects satisfy C_j 's preconditions. The current implementation uses a permissive rule: it returns compatible if at least one output-input match or at least one effect-precondition match exists. This is a candidate-link test, not a complete proof of executability.

For state execution, a capability is applicable when all of its preconditions match the current state and its required input values are available. Applying a capability updates the state with its effects and adds its outputs to the available-data set. A goal is satisfied when every required goal condition matches the final state.

6. Capability Composition Model

For an ordered sequence $Q = [C_1, C_2, \dots, C_n]$, the composition routine aggregates the capabilities into a composite record. Inputs produced by earlier capabilities are treated as internal dependencies; other inputs are exposed as external inputs. Preconditions already guaranteed by earlier effects are omitted from the external precondition set. Outputs, effects, and resources are accumulated across the sequence.

Operational attributes are combined using explicit assumptions. Total latency is the sum of individual latencies: $L(Q) = \sum L_i$. Composite reliability is estimated as $Rel(Q) = \prod Rel_i$, assuming independent success events. Composite risk is estimated as $Risk(Q) = 1 - \prod(1 - Risk_i)$, also assuming independence. Cost is summed: $Cost(Q) = \sum Q_i$. These calculations are illustrative and should be replaced by domain-specific aggregation rules when dependencies or shared resources affect performance.

The selected composite is `CreateOrder` → `MakePayment` → `SendNotification`. `CreateOrder` produces `order_id` and establishes `order.exists`. `MakePayment` consumes `order_id` and establishes `payment.completed`. `SendNotification` consumes `payment_status` and establishes `notification.sent`. In the simulation, the starting data includes `user_id`, `cart_items`, `payment_method`, and `message`, so the required inputs are available across the sequence.

7. Implementation

The implementation is written in Python and uses NumPy for vector operations, Pandas for tabular result export, Matplotlib for plots, and scikit-learn for cosine similarity. The main program defines the capability dataset, builds a common feature vocabulary, creates embeddings, checks compatibility, simulates execution, composes capabilities, runs experiments, and writes result files to a results directory.

- `encode_capability(cap)`: creates a multi-hot feature vector for an atomic capability.
- `encode_goal(goal)`: maps goal conditions to effect-style features.
- `cosine_score(a, b)`: calculates cosine similarity between two vectors.
- `is_compatible(a, b)`: checks for a matching output-input or effect-precondition link.
- `check_preconditions(cap, state)`: verifies the capability's state requirements.
- `execute_sequence(sequence)`: simulates the ordered workflow and records execution status.
- `compose(sequence)`: constructs a composite capability record and aggregates operational attributes.
- `goal_satisfied(state, goal)`: checks whether all goal conditions hold in the final state.

The program saves `alternative_implementations.csv`, `goal_relevance.csv`, `operational_attributes.csv`, `embeddings.json`, `summary.json`, and PNG plots in the results folder.

8. Experimental Methodology

The dataset contains eight manually specified capabilities: `CreateOrder`, `MakePayment`, `SendNotification`, `CancelCart`, `CreateOrderDB`, `CreateOrderGUI`, `RefundPayment`, and `TrackDelivery`. The initial state specifies authenticated user, non-empty cart, and existing cart conditions, while the goal requires order existence, completed payment, and a sent notification.

- **Capability compatibility**: evaluate selected capability pairs, including order creation followed by payment, payment followed by notification, order creation followed by delivery tracking, and cart cancellation followed by refund.
- **Capability composition**: compose and simulate `CreateOrder` → `MakePayment` → `SendNotification`, then inspect the composite record and final state.
- **Alternative implementations**: compare `CreateOrder`, `CreateOrderDB`, and `CreateOrderGUI`, which share functional effects but differ in resources and operational values.
- **Irrelevant capabilities**: calculate goal similarity for all eight capabilities to inspect whether directly goal-related capabilities receive higher scores.
- **Operational attributes**: compare latency and reliability across capabilities and calculate aggregate latency, reliability, risk, and cost for the selected sequence.

9. Results

The recorded run completed without a runtime error. The terminal output reported eight capabilities and an embedding vector size of 37. The selected sequence executed successfully in the simulator and the specified goal was reported as satisfied.

The aggregate values follow the program's chosen assumptions: latency and cost are summed, reliability is multiplied, and risk is calculated as one minus the product of individual survival probabilities. They are estimates based on the manually assigned capability attributes, not measured production-system performance.

The results directory contains the generated embedding JSON, summary JSON, CSV experiment tables, and plots for goal relevance, alternative implementations, latency, and reliability. The actual numerical similarity values should be taken directly from the generated CSV files when preparing the final submitted report.

10. Analysis

The prototype demonstrates that capabilities can be represented using a consistent numerical feature space and that a composite workflow can be described by aggregating the structures of its component capabilities. The symbolic simulation adds information that a similarity score alone cannot provide: the order-payment-notification sequence is checked against available inputs and current state preconditions, and the final state is compared with the goal.

Alternative order-creation implementations share inputs and effects but have different resource requirements and latency, reliability, and cost values. This illustrates why functional similarity and implementation identity should be treated separately. Goal relevance provides a first-pass ranking signal, but cosine similarity may be influenced by common features and does not establish that a capability is necessary or sufficient for a goal.

The implementation is computationally lightweight. If d is the vocabulary size, encoding a capability by iterating through its features is approximately $O(d)$ for vector initialization plus the number of features assigned. Cosine similarity for two dense vectors is $O(d)$. Storing m dense embeddings requires $O(md)$ numeric entries. These are asymptotic estimates; the current program does not benchmark runtime or memory empirically.

11. Limitations and Future Work

- The current dataset is manually specified and small; results may not generalize to other application domains.
- The feature-based embedding is not learned from data and does not capture semantic relationships beyond explicit shared feature labels.
- State and goal encoders should be exposed as explicit functions and evaluated independently as required by the assignment interface.
- Compatibility currently accepts one matching data or state link. A stronger checker should validate all required inputs, typed data compatibility, all preconditions, constraints, resources, and ordering.
- Capability-specific constraints, global policies, availability, and detailed execution mechanisms should be added to the dataset and representation.
- Composite capability vectors should be explicitly generated and compared with their atomic component vectors.
- Consistency should be tested on multiple formally specified application problems, not only the shopping example.
- Runtime and memory should be measured on increasing dataset and feature sizes.
- Reliability and risk aggregation assume independence; correlated failures and shared infrastructure are not modeled.

12. Conclusion

This assignment presents a transparent prototype for representing application capabilities as vectors and composing atomic operations into a larger workflow. The feature-based embedding captures explicit types, inputs, outputs, preconditions, effects, and resources, while cosine similarity offers a basic measure of vector relatedness. Symbolic compatibility and state simulation complement the embedding by checking whether the selected workflow can execute and reach its goal. The sample order-payment-notification sequence completed successfully under the defined dataset and assumptions. The prototype provides a foundation for capability composition, but a complete evaluation should extend state and constraint encoding, strengthen compatibility checking, test consistency across multiple problems, and measure computational efficiency.

Appendix A: Reproducibility

Requirements: numpy, pandas, matplotlib, scikit-learn. Create and activate a Python virtual environment, install dependencies using `python -m pip install -r requirements.txt`, and execute the program with `python main.py`. The results are written to the results directory. The virtual environment itself should not be committed to version control; it can be recreated from requirements.txt.